CASE STUDY · AI FOR DOCUMENTATION

Seven AI agents for the documentation lifecycle

From raw Slack threads and tickets to published, maintained docs, with a writer in charge at every step.

Debargho Ghosh · Technical Writer · 16 years of internal and external documentation

debarghoghosh86@gmail.com · linkedin.com/in/debargho-ghosh-8566ab23

1 Intake	2 Planning	3 Drafting	4 Review	5 QA	6 Publishing	7 Maintenance
Sorts raw material; builds source packs	Checks existing docs; writes briefs	Drafts from sources; tags every fact	Checks the shared house style	Checks the whole release	Gates, cleans, and prepares the PR	Finds drift; routes fixes back

Six human checkpoints: confirm scope · approve plan · edit draft · sign off release · approve merge · approve maintenance tickets

The challenge

Documentation work is mostly judgment: sorting decided from discussed, catching what's missing, keeping dozens of articles consistent, and knowing what's safe to publish. I wanted to find out how much of that an AI pipeline could do reliably, and what a senior writer has to put into it to make it trustworthy.

What I built

Seven specialized agents in Claude Projects, one per stage of the doc lifecycle. Each has written instructions, rules, and an output format that the next stage can use directly. Key design choices:

- **Traceable facts.** Every drafted fact carries a source tag. Missing facts become [GAP] markers and SME questions, never guesses.
- **Shared contracts.** Agents agree on markers, source pack format, and one numbered question list (I-Q#, P-Q#, D-Q#), so nothing is asked twice.
- **Single source of truth.** One house-style.md, used by four agents. A rule changed once applies everywhere, the same idea as DITA reuse.
- **Strict gates.** QA gives release verdicts. The publisher may only remove tags and build links; everything else goes back to a human.
- **A closed loop.** Maintenance groups post-release drift into change sets and routes them back into the pipeline.

How I worked

- **Answer keys first.** Every stage was tested on realistic inputs with planted traps: reversed decisions, Won't Fix tickets, duplicate articles, wrong link titles, claimed-but-missing fixes.
- **Evaluation loop.** Run, score, find the failure, change the instructions, check for side effects downstream, run again. 21 instruction versions in total.
- **Real legacy content.** I tested the reviewer on a help page I wrote years ago, reviewed as HTML so it could check alt text and links.

What my writing experience brought

- A severity scale for review, set by editorial judgment (passive voice and missing alt text: High).
- Editor's habits turned into rules: one comment per sentence; don't fix what you've queried; keep a title so links don't break.
- Knowing when a run is good enough, and when variation needs a human check.

Results

27 → 0
Style issues: a legacy hand-written page vs. the pipeline's final draft

0
Invented facts across all drafting runs, with every fact tagged to a source

10 / 10
Planner: all planted structure traps, including a duplicate article

10 + 1
QA: all planted release defects, plus one I hadn't planted; no false alarms

5 / 5
Maintenance: articles affected by one UI rename, found in a single sweep

6 / 6
Publisher: not-ready items held back, including a fix claimed but not made

What the agents caught that I didn't

- Screenshots in my old help page that appeared to be out of order.
- A missing link in a release draft that wasn't in my answer key.
- Approved release notes that left out a UI rename shipping in the same version.

Lessons I'd bring to a team

- **The writer's expertise is the product.** The model knew the style rules; it needed an editor's judgment about output and process.
- **Every instruction has side effects.** One fix removed the version number two stages later. Test the whole pipeline.
- **Check the release, not just the article.** The worst defects live between documents.
- **Verify, don't trust.** Gates should check claims against the text.

Skills demonstrated

AI agent and prompt design · AI output evaluation · information architecture · content strategy · docs-as-code conventions (front matter, pull requests, redirects) · content governance and release management · version-aware docs · accessibility (WCAG alt text and link text) · Microsoft Writing Style Guide · SME collaboration

Next

Connect the stages with a workflow tool (n8n or Make), then rebuild in Python with AI-assisted coding and run it from GitHub pull requests, measured across a larger test set.

Full case study (README), all 21 agent instruction versions, test inputs with answer keys, and a 49-page learning journal are available on request.
Built in Claude Projects. Product names in test material are fictional (Contoso Charts).